Sicurezza dei Sistemi Informativi

01UDUOV

Politecnico di Torino

Prof. Cataldo Basile

A cura degli studenti:
A.A. 2025/26: *Alessandro Annecchini*, *Alessio Giliberti*, *Chiara Mercurio*, *Luca Giacchetta*

Copyright © 2025-2026 Cataldo Basile
Licensed under the Creative Commons Attribution-NonCommercial 3.0 Unported License (the "License"). You may not use this file except in compliance with the License. You may obtain a copy of the License at `http://creativecommons.org/licenses/by-nc/3.0`. Unless required by applicable law or agreed to in writing, software distributed under the License is distributed on an "AS IS" BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied. See the License for the specific language governing permissions and limitations under the License.

Indice

1 Gestione dei rischi informatici 5

1.1 Fondamenti e Terminologia del Rischio 5

1.2 Vulnerabilità e CVE – Common Vulnerabilities and Exposures 5

1.3 CWE e Window of Exposure 6

1.3.1 Esempio di Vulnerabilità Critica: Meltdown (CVE-2017-5754) 6

1.3.2 EPSS – Exploit Prediction Scoring System 6

1.4 Window of Exposure 7

1.5 Risk Management Framework (RMF) 8

1.6 Vulnerability Assessment 9

1.6.1 Vulnerability Assessment tools 9

1.7 Valutazione del Rischio 11

1.8 Vulnerability Management e Patch Management 13

1.8.1 Ruoli e Strategie 14

1.8.2 Affidabilità delle Patch e Problematiche 16

1.8.3 Evoluzione del Modello di Rilascio delle Patch 16

1.8.4 Enterprise Patch Management Tools 17

Todo list

1. Gestione dei rischi informatici

La gestione del rischio informatico (*Risk Management*) si configura come un'attività essenziale in ogni azienda, con l'obiettivo primario di **quantificare i rischi e ridurli entro limiti tollerabili** tramite l'applicazione di un **processo ripetibile e oggettivo**. Questo approccio metodologico è necessario per minimizzare i costi diretti e indiretti derivanti da incidenti e attacchi. Ma diventa indispensabile nel caso si fosse interessati ad aderire a standard (come la famiglia ISO/IEC 27000) e normative (come il GDPR) che richiedono esplicitamente un'analisi del rischio. Il processo deve essere il più possibile scientifico e rigoroso per evitare che la valutazione del rischio sia arbitraria o soggettiva e, in ultima istanza, inutile.

1.1 Fondamenti e Terminologia del Rischio

Il processo di gestione del rischio richiede la precisa definizione e distinzione dei seguenti elementi:

- **Asset**: qualsiasi risorsa che abbia valore per l'azienda e sia necessaria all'erogazione di un servizio IT, inclusi beni materiali (es. server) e immateriali (es. reputazione, proprietà intellettuale).
- **Debolezza (*Weakness*)**: una predisposizione del sistema a manifestare problemi, come codice scritto male o configurazione non attenta.
- **Vulnerabilità (*Vulnerability*)**: una debolezza scoperta e **sfruttabile direttamente dagli attaccanti** per violare le proprietà di sicurezza di un sistema o di una rete.
- **Minaccia (*Threat*)**: Un evento (intenzionale o accidentale) che, se sfruttato, può causare la perdita di una o più proprietà di sicurezza (es. Confidenzialità, Integrità, Disponibilità).
- **Attacco**: il verificarsi di una minaccia di tipo intenzionale.
- **Evento Negativo**: il verificarsi di una minaccia di tipo accidentale (es. guasto hardware o errore umano).

1.2 Vulnerabilità e CVE – Common Vulnerabilities and Exposures

Le vulnerabilità di sicurezza informatica note e rese pubbliche sono descritte in modo standardizzato nel database **Common Vulnerabilities and Exposures (CVE)**, mantenuto dalla **MITRE Corporation**. Le CVE riportano un insieme standard di attributi che servono a caratterizzarle:

- **CVE-ID**: ogni vulnerabilità ottiene un ID univoco (es. CVE-2017-5754), una descrizione testuale che e una serie riferimenti per approfondimenti. L'ID è composto da una stringa fissa (CVE), l'anno in cui la CVE è stata resa pubblica, e un numero incrementale azzerato all'inizio di ogni anno.
- **CVSS (Common Vulnerability Scoring System)**: è un punteggio da **0 a 10** che fornisce un'idea del **danno potenziale** che lo sfruttamento della CVE può causare. Questo punteggio è calcolato mediante una formula standard a partire dai valori di **metriche base** che vengono esplicitamente associati alla CVE:
  - *Attack Vector*: da dove può essere lanciato l'attacco (ad esempio, accesso fisico, rete locale o remoto).

  - *Attack Complexity* e *Privileges Required*: La difficoltà che un attaccante può incontrare nello sfruttamento e le competenze che servono, e i permessi necessari allo sfruttamento (da nessun privilegio a privilegi di root).
  - *Impatto*: l'impatto potenziale descritto da tre metriche distinte per Confidenzialità, Integrità e Disponibilità (CIA).

Le vulnerabilità sono classificabili come **Non note** (se ancora nessuno l'ha scoperta) o **Note**; in quest'ultimo caso si può specializzare la classificazione in:

- *zero-day*, da quando viene scoperta (da un hacker o un ricercatore) fino a quando non viene pubblicata una *patch*.
- *unpatched*, da quando viene pubblicata la patch a quando viene installata sul dispositivo vulnerabile.
- *patched*, successivamente all'installazione della patch.

Una **patch** è una porzione di codice che rende innocua una vulnerabilità [1].

1.3 CWE e Window of Exposure

La **CWE (Common Weakness Enumeration)** è una tassonomia che cataloga gli errori di programmazione, configurazione e gestione che potrebbero essere rilevati nel software e nell'hardware. Queste debolezze devono essere monitorate da chi sviluppa il codice poiché potrebbero trasformarsi in vulnerabilità sfruttabili (CVE). Un esempio comune è la **CWE-20** (*Improper Input Validation*). Non c'è la certezza che una debolezza riuscirà ad essere sfruttata, ma di sicuro va monitorata ed eliminata se possibile per anticipare i problemi.

1.3.1 Esempio di Vulnerabilità Critica: Meltdown (CVE-2017-5754)

Un esempio notevole di vulnerabilità catalogata è **Meltdown** (CVE-2017-5754)[1][2][3]. Questa vulnerabilità è associata ai sistemi dotati di microprocessori (Intel) che utilizzano l'**esecuzione speculativa** e la predizione di salto (*indirect branch prediction*) per ottimizzare le prestazioni. Sfruttando questa tecnica, un attaccante con accesso fisico all'apparecchiatura può eseguire un attacco di **side-channel** della *data cache* per avere accesso non autorizzato ad informazioni riservate. Meltdown è correlata alla debolezza del codice **CWE-200** (*information exposure*). Le mitigazioni software/microcodice introdotte per correggere Meltdown hanno portato a una **riduzione drastica delle performance** (anche oltre il 30%) in molti processori affetti.

1.3.2 EPSS – Exploit Prediction Scoring System

Per completare le informazioni su quanto possono essere pericolose le vulnerabilità per un'azienda interessata a svolgere un'analisi dei rischi, per completare il punteggio CVSS, che descrive il massimo danno, è stato introdotto l'**EPSS (Exploit Prediction Scoring System)**. Questo parametro stima la **probabilità** che la vulnerabilità possa essere sfruttata nel mondo reale in un determinato momento basandosi su dati dinamici e osservazioni ottenute (*in the wild*), cioè dati reali raccolti da Internet. L'organizzazione nonprofit FIRST (Forum of Incident Response and Security Teams) mantiene aggiornate le statistiche ed i punteggi EPSS.

Esempio: EPSS di Meltdown

Considerando la vulnerabilità Meltdown (CVE-2017-5754), è interessante osservare come l'EPSS sia variato nel tempo. Al momento della divulgazione pubblica (gennaio 2018), il punteggio EPSS era relativamente basso (circa `0.02`, ovvero probabilità del 2% di sfruttamento nei successivi 30

[1] NIST, 2017
[2] MITRE, 2017
[3] Google Project Zero, 2017

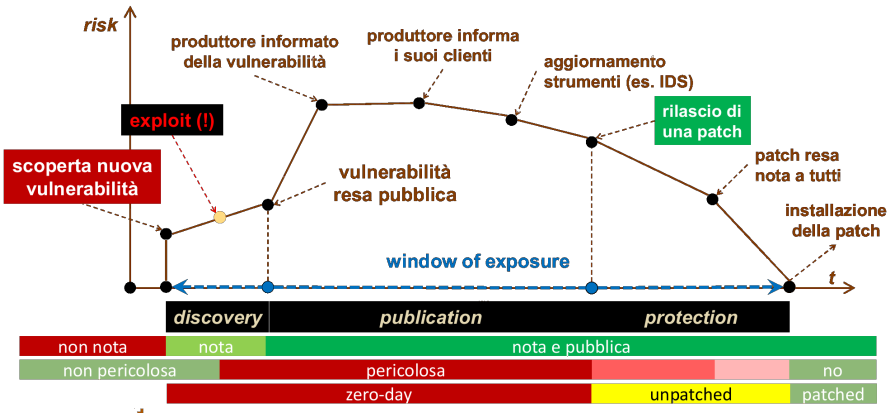


Figura 1.1: Finestra di esposizione

giorni) nonostante l'elevato CVSS di 5.6, poiché lo sfruttamento richiedeva competenze tecniche avanzate e accesso locale al sistema. Tuttavia, con la pubblicazione di *proof-of-concept* pubblici e l'aumento della consapevolezza nella comunità di sicurezza, l'EPSS è gradualmente aumentato nelle settimane successive, raggiungendo picchi superiori al 5% quando sono stati rilasciati tool automatizzati per lo sfruttamento. Dopo la disponibilità delle patch e delle mitigazioni software, l'EPSS è progressivamente diminuito, stabilizzandosi attorno allo 0.5% nei mesi seguenti. Questo andamento dimostra come l'EPSS catturi dinamicamente l'evoluzione del rischio reale, fornendo informazioni complementari al CVSS statico e permettendo una prioritizzazione più accurata degli interventi di patching.

1.4 Window of Exposure

Il concetto di **window of exposure** (finestra di esposizione) definisce l'intervallo temporale ad alto rischio che intercorre tra la **scoperta di una vulnerabilità** e la sua **correzione**.

- La vulnerabilità passa dallo stato *non nota* a *nota* quando viene scoperta.
- Diventa **pericolosa** quando viene trovato un **exploit**, cioè un pezzo di codice che la sfrutta. Un exploit dimostra che si tratta in effetti di una vulnerabilità.
- Il rischio aumenta drasticamente quando la vulnerabilità viene **resa pubblica** (disclosure), perché il numero di attaccanti potenzialmente in grado di trovare exploit e sfruttarla aumenta.
- Durante l'attesa del rilascio della patch, è necessario aggiornare gli strumenti di monitoring (es. IDS – Intrusion Detection System) per rilevare tentativi di sfruttarla.
- Secondo il diagramma proposto, dopo il rilascio della patch, il livello globale di rischio cala, fino al momento in cui la patch viene installata su tutti i dispositivi affetti. In realtà, qualche studioso propone una visione diversa. Infatti, una volta rilasciata una patch, gli attaccanti possono farne reverse engineering, scoprire il problema, la vulnerabilità e preparare un exploit più facilmente. È quindi importante applicare la patch prima possibile perché dopo il rilascio il numero di attaccanti in grado di poterla sfruttare potrebbe aumentare.
- Le vulnerabilità **zero-day** sono quelle scoperte per le quali non esistono ancora patch. Si può venire a conoscenza delle zero-day sul mercato illegale nel dark web oppure perché sono state rese pubbliche tramite canali ufficiali (di solito dopo la disponibilità di una patch).

L'organizzazione **Zero Day Initiative (ZDI)** propone una gestione responsabile delle vulnerabilità, operando come intermediario tra i ricercatori di sicurezza e i fornitori di software. Una volta scoperta una vulnerabilità, ZDI notifica privatamente il vendor interessato, stabilendo una *deadline* di 120 giorni per il rilascio di una patch correttiva. Qualora il fornitore non rilasci la correzione entro tale termine, ZDI procede con la divulgazione pubblica della vulnerabilità (*full disclosure*),

con conseguenze negative per la reputazione del vendor e un incremento sostanziale del rischio per gli utenti del software vulnerabile.

1.5 Risk Management Framework (RMF)

Il modello di riferimento consigliato dal **NIST** per la gestione dei rischi è il **RMF**, articolato in quattro fasi:

1. **Risk Framing (Preparazione)**. Questa fase iniziale stabilisce il contesto e i presupposti per l'intero processo di gestione del rischio. Si stabilisce l'ambito del sistema da proteggere, definendo chiaramente i confini organizzativi e i dettagli tecnologici. Si identificano gli asset materiali e immateriali (come la reputazione aziendale) e si determina il loro valore per l'organizzazione. L'output principale è la descrizione dettagliata della **metodologia di valutazione** che verrà applicata, includendo le metriche da utilizzare e tutte le formule per il calcolo dei rischi, i criteri per caratterizzare quando dei rischi sono accettabili e le strategie di gestione. La metodologia definita durante il framing deve essere **ripetibile** per garantire coerenza nel tempo e tra diverse valutazioni, oltre a preservare il *know-how* organizzativo anche in caso di turnover del personale tecnico.
2. **Risk Assessment (Valutazione)**. In questa fase si procede all'identificazione sistematica delle minacce (sia intenzionali che accidentali), delle debolezze architetturali e implementative, e delle vulnerabilità tecniche (interne ed esterne al perimetro aziendale). Per ciascun rischio identificato si stimano gli **impatti** potenziali, che possono includere danni finanziari diretti e indiretti, interruzione o blocco dei processi critici, perdita di reputazione, e conseguenze legali o regolamentari. Parallelamente, si valuta la **probabilità** di accadimento (*likelihood*) di ogni scenario di rischio, considerando fattori quali la presenza di controlli esistenti, le capacità degli attaccanti, e l'attrattività del target. L'obiettivo è valutare tutti (o la maggior parte) dei rischi rilevanti, producendo una mappatura completa del profilo di rischio delle parti dell'organizzazione, coerentemente con quanto deciso in fase di framing.
3. **Risk Response/Mitigation (Risposta e Mitigazione)**. Sulla base dei rischi identificati e quantificati, si individuano e valutano le possibili soluzioni di mitigazione. Ogni soluzione viene analizzata in termini di **costo** di implementazione e mantenimento, **tempo necessario** per la realizzazione e messa in opera, e **impatto** effettivo sulla riduzione del rischio (sia in termini di probabilità che di conseguenze). La valutazione deve considerare diverse alternative, tra cui l'accettazione del rischio (quando il costo della mitigazione supera il danno potenziale), il trasferimento del rischio (ad esempio tramite assicurazioni), l'annullamento del rischio (es. eliminando l'attività che lo genera), o la sua riduzione attraverso controlli tecnici e organizzativi. La decisione finale deve essere motivata oggettivamente e documentata, anche nel caso si scelga di accettare o ignorare il rischio. Una volta selezionate le contromisure, si procede all'implementazione delle soluzioni scelte secondo le priorità definite.
4. **Risk Monitoring (Monitoraggio)**. Questa fase non è conclusiva ma continuativa. Essa valuta l'efficacia delle mitigazioni implementate sul campo, verificando che i controlli funzionino come previsto e raggiungano effettivamente gli obiettivi di riduzione del rischio. Si controlla la validità delle assunzioni fatte nelle fasi precedenti, poiché il contesto operativo, il panorama delle minacce e le tecnologie evolvono costantemente. Il monitoraggio include la verifica della conformità alle policy di sicurezza, l'analisi degli indicatori di compromissione, e la raccolta di metriche sull'efficacia dei controlli. L'obiettivo è mantenere il processo aggiornato e adeguato ai cambiamenti del panorama delle minacce e dell'infrastruttura IT. Quando si rilevano scostamenti significativi, modifiche architetturali, nuove minacce o inefficacia delle mitigazioni, il processo viene iterato: si può ricominciare dal *Risk Framing* per una revisione completa, oppure tornare a una fase precedente specifica per aggiustamenti mirati, mantenendo così il profilo di gestione del rischio sempre aggiornato.

1.6 Vulnerability Assessment

Il **Vulnerability Assessment** è un processo che valuta l'esposizione aziendale alle vulnerabilità, focalizzandosi prevalentemente sulle minacce di rete. Esso non è da considerarsi una fase indipendente dell'RMF, è infatti un'attività che potrebbe essere necessaria come sottofase del Risk Assessment. Il Vulnerability Assessment consiste in quattro fasi:

1. **Planning**. Definire lo *scope* (cosa valutare) e lo *schedule* (quando svolgerlo).
2. **Information Gathering**. Acquisire informazioni sugli obiettivi (topologia, host, software installati). Per quanto riguarda gli host, è fondamentale identificare il sistema operativo e la loro versione per associare le vulnerabilità note.
3. **Scanning**: Cercare vulnerabilità note (CVE). Le tecniche di scansione includono:
   - *host discovery*[4]: ad esempio con ICMP echo request, ARP request, TCP SYN, UDP.
   - *port scanning*: per classificare le porte in *open* (c'è un servizio raggiungibile in ascolto), *closed* (non in ascolto), *filtered* (non raggiungibile, ad es. un firewall blocca la connessione).
   - *OS detection*: alcuni scanner sono in grado di determinare il sistema operativo (e alcune volte la versione) tramite delle euristiche in base a come gli host rispondono a determinate sequenze di pacchetti.
   - *TCP Sequence Predictability Classification*: lo scanner cerca di determinare il prossimo sequence number che userà il TCP, per testare la bontà del generatore di numeri casuali; in caso sfavorevole, un attacco di tipo spoofing potrebbe essere attuabile.
4. **Report**: Stesura di un documento che presenti i risultati e includa le proposte per le mitigazioni in base a vari fattori (la migliore, la più conveniente e quella suggerita).

1.6.1 Vulnerability Assessment tools

Esistono dei software in grado di automatizzare questo processo, almeno nei punti 3 e 4: difatti, questi programmi si occupano di trovare i target raggiungibili, identificare e categorizzare le vulnerabilità e produrre dei report complessivi. Tra questi tool, alcuni tra i più famosi sono:

- **nmap**, CLI (*Command-Line Interface*) può essere usato anche da attaccanti per effettuare qualunque tipo di scansione; ha una documentazione molto ricca, ma per usarlo in maniera consistente ed esaustiva in ambienti aziendali, è necessario creare script che automatizzino vari tipi di ricerche e che supportino scansioni in diverse parti della rete aziendale. Inoltre, genera report molto sintetici o disorganizzati. È particolarmente adatto per scansioni mirate e personalizzate, ma risulta meno pratico in scenari aziendali complessi.
- **GVM (OpenVAS)**, è una soluzione open source per l'automazione dello scanning in ambienti aziendali. Rappresenta una soluzione integrata che combina discovery, scanning e reporting in un'unica piattaforma, particolarmente indicata per ambienti enterprise con reti articolate e numerosi asset da monitorare. La versione free è accessibile a tutti, ma se si vogliono identificare le vulnerabilità più recenti o quelle per sistemi non Linux è necessario pagare. Esistono anche delle *appliance*, più costose, pensate per facilitare le scansioni.
- **Nessus**, una volta il più utilizzato, anche perché open source e gratis. Dopo averlo reso chiuso e a pagamento (anche se si può installare e si possono lanciare alcune scansioni gratis), è stato creato OpenVAS come suo fork. Al contrario delle prime versioni, che permettevano di lanciare gli exploit solo premendo un tasto, le versioni più recenti e i loro fork hanno disabilitato le funzioni per evitare danni fatti da script kiddy[5]. Offre funzionalità avanzate di

[4] questo passo viene spesso associato anche all'Information Gathering

[5] Script kiddy è un termine dispregiativo usato per indicare individui con scarse competenze tecniche che utilizzano strumenti automatizzati sviluppati da altri per condurre attacchi informatici, spesso senza comprendere i meccanismi sottostanti né le conseguenze delle loro azioni.

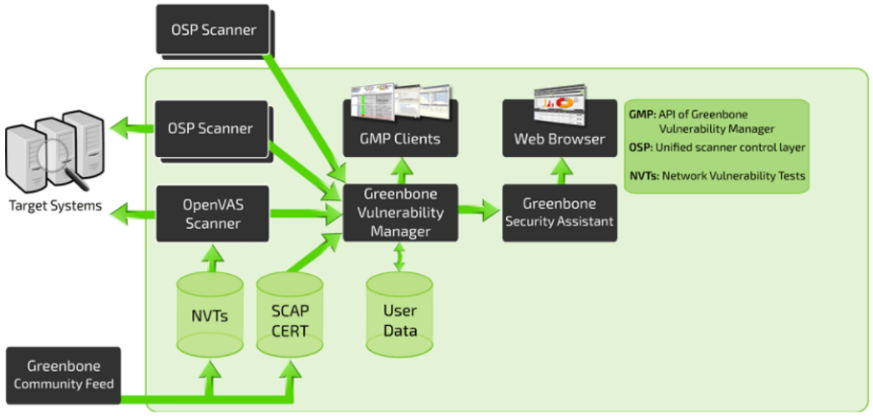


Figura 1.2: Architettura di GVM (Fonte: Medium)

gestione centralizzata e reportistica, rendendolo appropriato per organizzazioni di medie e grandi dimensioni.

- **Acunetix**, software proprietario, completo, molto rifinito anche nelle interfacce grafiche e quindi molto costoso, che offre anche un'analisi approfondita delle vulnerabilità delle applicazioni web, con capacità di scanning e correlazione automatica dei risultati.

La scelta tra questi strumenti dipende dalle dimensioni e dalla complessità dell'infrastruttura da proteggere: mentre **nmap** è eccellente per analisi puntuali e test di sicurezza mirati, i tool integrati come **GVM** e **Nessus** sono più adatti ad ambienti enterprise con reti vaste e complicate, dove è necessario gestire sistematicamente migliaia di dispositivi, consolidare i risultati da multiple sorgenti, mantenere database storici delle vulnerabilità, e generare report conformi agli standard di compliance. Questi strumenti integrati offrono inoltre funzionalità di orchestrazione, scheduling automatico delle scansioni, gestione centralizzata delle policy, oltre ad integrarsi con i sistemi di ticketing e workflow aziendali.

Esempio: OpenVAS

In Figura 1.2 viene presentata l'architettura di **OpenVAS** (Open Vulnerability Assessment System), che recentemente ha cambiato nome in GVM. GVM è composto da più componenti modulari che collaborano per eseguire la scansione e l'analisi delle vulnerabilità di sistemi e reti. Gli *OpenVAS Scanner*, sono i componenti responsabili dell'esecuzione effettiva dei test di vulnerabilità e per questo motivo devono essere posizionati in maniera tale da riuscire a svolgere le scansioni senza interferenze (es. nella stessa sottorete per evitare i filtri imposti dai firewall). In contesto GVM, i test sono definiti Network Vulnerability Tests (NVTs), e sono basati sul linguaggio NASL (Nessus Attack Scripting Language). Questi test sono forniti dal *Greenbone Community Feed*, un repository costantemente aggiornato di script di scansione, sebbene gli aggiornamenti più recenti siano a pagamento.

In alternativa agli scanner nativi OpenVAS, è possibile collegare altri tool di scansione; l'importante è che possano essere configurati tramite il protocollo *OSP (Open Scanner Protocol)*.

La gestione e il controllo delle attività di scansione sono demandati al *Greenbone Vulnerability Manager*, che coordina l'esecuzione delle scansioni interfacciandosi con gli scanner perché analizzino i vari target, e gestisce la memorizzazione dei risultati. Il *Greenbone Security Assistant (GSA)* è il server che espone un'API che permette la configurazione del Greenbone Vulnerability Manager mediante un'interfaccia web. Tramite browser gli utenti possono configurare le scansioni, visualizzare i report e gestire i risultati in modo interattivo. Tutti questi componenti comunicano tramite protocolli dedicati, in particolare l'*OpenVAS Management Protocol (OMP)* e il *Greenbone Management Protocol (GMP)*, che garantiscono un'interazione sicura e strutturata tra le diverse

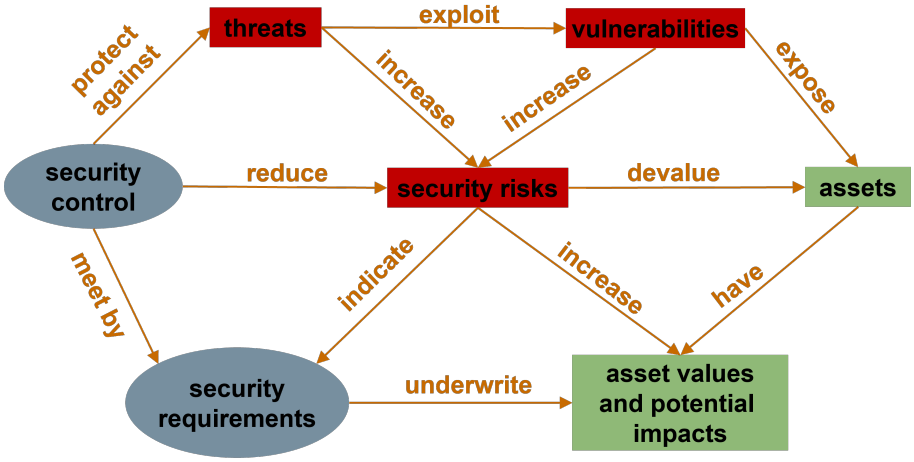


Figura 1.3: Un possibile modello per rappresentare le relazioni tra i concetti legati al rischio

entità del sistema. L'architettura modulare consente un'elevata scalabilità e flessibilità, rendendo OpenVAS adatto sia a piccoli ambienti di test che a infrastrutture di sicurezza di livello enterprise.

Infine, le funzionalità del Greenbone Vulnerability Manager sono anche accessibili in maniera programmatica tramite una libreria Python; è quindi possibile usare dei *GMP Client* dedicati ed automatizzare tramite script la gestione delle scansioni ed il processamento dei risultati.

1.7 Valutazione del Rischio

Dare un peso al rischio è necessario anche per *stabilire un ordine di priorità* di intervento. L'obiettivo è sempre ridurre i rischi a livelli accettabili applicando mitigazioni. Tre aspetti sono stati considerati fondamentali per la valutazione del rischio:

- Il *danno* massimo (e medio) in caso di manifestazione del rischio e le categorie di asset interessate. Questo parametro è importante perché permette di valutare le conseguenze peggiori che potrebbero verificarsi nel caso il rischio si manifestasse. Serve per valutazioni sia finanziarie che di funzionamento delle infrastrutture.
- I *tempi di recupero* della piena funzionalità (in caso di attacco). Questa metrica è anche utile per stimare le conseguenze indirette, es. i mancati guadagni o i problemi al personale che non potrà lavorare.
- La *probabilità* che esso si verifichi. Naturalmente, essendo stime probabilistiche, non tutti i rischi si manifesteranno, ma è importante sapere se sono rischi concreti (probabilità alte) oppure sono fenomeni che potrebbero accadere con probabilità talmente trascurabile che potrebbe essere anche opportuno accettarli.

La valutazione può essere *quantitativa*, se ad ogni rischio viene associato un numero, ad esempio come risultato del calcolo di una formula. Anche se i numeri vengono rappresentati con molta precisione, va ricordato che tali valori devono essere valutati in termini di ordini di grandezza (un punteggio di 5.55 non indica un rischio davvero maggiore di un altro valutato 5.33, mentre è sicuramente più basso di un rischio valutato 112.34). La valutazione può anche essere *qualitativa*, se i rischi vengono associati a un numero limitato di categorie (es. 3 valori ordinati tipo low, moderate, high). Esiste anche la valutazione *semi-quantitativa* se i rischi vengono associati a valori numerici indicativi di categorie qualitative; ad es. 1:low, 2:moderate, 3:high.

I numeri ottenuti dall'approccio quantitativo sono spesso ingannevoli, quindi si preferisce mapparli su approcci semi-quantitativi, analoghi a quelli qualitativi, ma con il vantaggio di poter lavorare con i numeri, rendendo possibile definire delle formule.

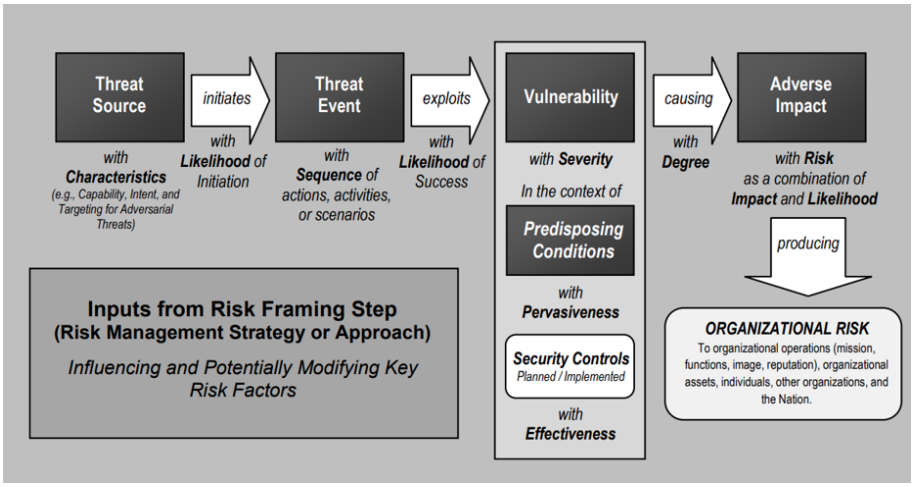


Figura 1.4: Esempio di modello di rischio (Fonte: NIST RMF)

In passato, si è pensato di usare anche l'approccio della *monetization*. Questo approccio proponeva di associare un costo ai rischi, invece di un punteggio numerico. Di solito, questo punteggio indicava il valore medio (e spesso anche quello massimo) associato al rischio (che quindi considera anche la probabilità di accadimento). Tale approccio è stato proposto per rendere la valutazione immediatamente comprensibile (o spaventare) chi, a livello di business, doveva decidere se finanziare spese per la mitigazione dei rischi informativi.

Per calcolare il valore finale associato ai rischi, lo *score*, sono stati proposti tre approcci principali:

- *Threat-Oriented* (cfr. Figura 1.4): Si parte dalle minacce, che vengono caratterizzate ad esempio da obiettivi, competenze, budget. Dalla caratterizzazione delle minacce si può capire che operazioni sarebbero in grado di svolgere (*threat event*) e con quale probabilità. Dagli eventi si può capire quali vulnerabilità potrebbero sfruttare anche in base alle debolezze (*predisposing conditions*). A partire da questi dati si può stimare l'impatto ed il rischio tecnico (tipicamente come impatto $\times$ probabilità che si verifichi), che deve successivamente essere trasformato in un rischi organizzativo globale. In questo modo, quello proposto dal NIST nel RMF, è possibile valutare l'impatto delle mitigazioni (*security control*), sia quelli già usati sia quelli che si deve valutare di applicare per portare il rischio a livelli tollerabili.
- *Asset/Impact-Oriented*: facendo riferimento alla Figura 1.3, si parte dagli asset critici (*mission/business critical*) e si valutano le conseguenze della loro compromissione. Successivamente si identificano le minacce che potrebbero essere interessate a comprometterli, le loro capacità, budget, in modo da poter stimare le probabilità che riescano a sfruttare. Sulla base di queste informazioni si calcolano i rischi e si valutano le mitigazioni. In sostanza, si parte da punti diversi, ma è equivalente a quello threat-oriented.
- *Vulnerability-Oriented*: metodo un po' naif e semplificato, parte dall'identificazione delle vulnerabilità già identificate (es. tramite *Vulnerability Assessment*) e, usando punteggi standard come EPSS e CVSS, valuta l'*exploitability* per stimare costi, probabilità e conseguenze. Chi adotta questo approccio non sempre poi si impegna a identificare le parti interessate a volerle sfruttare, capacità o motivazioni. Per questo motivo, questo metodo è meno rigoroso e più approssimativo, ma veloce e meno costoso.

Strumenti come il *MITRE ATT&CK*[6] forniscono una *knowledge base* delle principali tecniche di attacco per costruire *threat model* affidabili e completi: è una grande tassonomia di attacchi (divisi in 14 macrocategorie) con migliaia di esempi, con riferimenti su come mitigare i rischi. Analogamente,

[6] MITRE, 2013

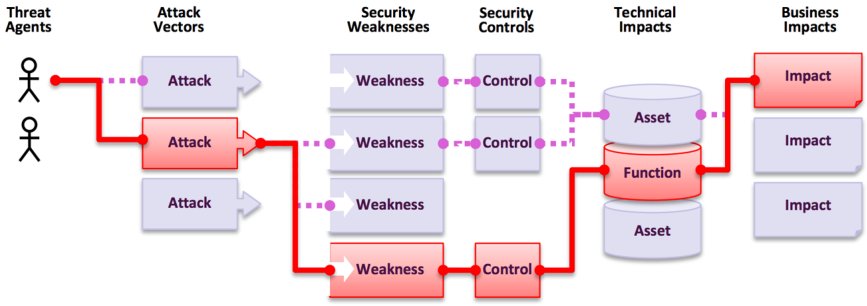


Figura 1.5: Esempio di modello di rischio (Fonte: OWASP)

c'è il *MITRE D3FEND*[7][8] che costituisce una base di conoscenza per tutte le contromisure attraverso dei grafi, a partire da quasi un migliaio di *artefatti* (asset, oggetti o tipologie), che si collegano agli attacchi, piattaforme ed eventuali contromisure. Questi modelli sono estremamente utili per aiutare a svolgere l'analisi dei rischi e stimare il livello di copertura (e quindi affidabilità) delle fasi di assessment.

L'ente *OWASP (Open Web Application Security Project)* fornisce diverse risorse su come effettuare l'assessment delle proprie web app e delle Top 10 periodiche in cui si specificano i rischi più alti per le applicazioni web (recentemente anche per l'uso degli LLM). OWASP utilizza un approccio semi-qualitativo per la valutazione dei rischi nelle applicazioni web, calcolando il punteggio come likelihood $\times$ impact. L'approccio proposto è di tipo threat-oriented, in cui le fasi seguono un flusso standard, come descritto dal grafico Figura 1.5. Si parte dalle minacce (i *threat agent*) che sono in grado di lanciare determinati attacchi, tra una serie di attack vector noti, che sfruttano debolezze (*security weaknesses*) del sistema. Questi attacchi causano impatti a livello tecnico che si ripercuotono anche a livello di business. I security control, già applicati o da implementare come mitigazione, riducono le conseguenze tecniche.

1.8 Vulnerability Management e Patch Management

Il **Vulnerability Management** è il processo aziendale continuo che mira a minimizzare la *window of exposure*. Partendo dall'assunzione che il software ha sicuramente delle debolezze e spesso delle vulnerabilità note. L'obiettivo di questo processo è **automatizzare la gestione delle vulnerabilità**, in modo da affrontare il problema delle vulnerabilità in maniera organizzata, con l'obiettivo ultimo di minimizzare il rischio informativo, lasciando il sistema esposto alle vulnerabilità il minor tempo possibile e, in caso di vulnerabilità multiple, eliminare prima quelle che potrebbero creare i danni maggiori.

L'approccio proposto dal NIST, nel documento IR 8011 Vol. 4, propone di definire un *desired state*, cioè uno stato di esposizione ai rischi dati dalle vulnerabilità caratterizzato da metriche ben precise, ad esempio, il massimo numero di vulnerabilità per classe di CVSS che possono essere accettate, la durata massima del processo di patching (da quando una vulnerabilità diventa nota a quando verrà corretta). Questo desired state deve poi essere confrontato con *actual state*, mantenuto sempre aggiornato mediante strumenti di vulnerability assessment. L'uso del **Information Security Continuous Monitoring (ISCM)** è essenziale per accorgersi subito di nuove vulnerabilità.

Il **Patch Management** è il processo specifico di identificazione, acquisizione, installazione e verifica delle patch. È un compito di sicurezza fondamentale per mitigare i rischi. Anche in questo caso, un documento di riferimento è il NIST SP 800-40, "Guide to Enterprise Patch Management".

[7] MITRE, 2013
[8] MITRE, 2013

1.8.1 Ruoli e Strategie

Il NIST sottolinea che ogni task, specie quelle essenziali per la sicurezza, richieda che ci sia personale dedicato e che le attività da svolgere siano ben definite. A tal fine individua dei ruoli specifici per gestire questo processo:

- **Software Flaw Manager (SFM)**: questo ruolo è stato proposto per affrontare, a livello di sviluppatori e venditori di software, tutte le problematiche relative a bug e problemi di sicurezza. È tipicamente ricoperto dal personale che ha maturato molta esperienza nello sviluppo e gestione di software e ha la responsabilità di coordinare l'intero ciclo di vita delle vulnerabilità dal punto di vista dello sviluppo. Le sue attività principali includono:
  - Coordinare i processi di *discovery* delle debolezze, sia attraverso audit interni del codice che tramite programmi di *bug bounty* o collaborazioni con ricercatori di sicurezza esterni.
  - Gestire la ricezione e la validazione delle segnalazioni di vulnerabilità provenienti da fonti esterne (es. tramite programmi di *responsible disclosure*).
  - Supervisionare lo sviluppo delle patch correttive, coordinando i team di sviluppo per garantire che le correzioni siano efficaci e non introducano nuove problematiche.
  - Stabilire le priorità di sviluppo delle patch in base alla gravità delle vulnerabilità e all'impatto potenziale sugli utenti.
  - Comunicare tempestivamente con i clienti e le organizzazioni di sicurezza riguardo alle vulnerabilità scoperte e alle patch disponibili.

  La presenza di questo ruolo è fondamentale per garantire che il processo di gestione delle vulnerabilità sia strutturato e non lasciato alla casualità, evitando ritardi pericolosi nel rilascio delle correzioni che hanno di solito impatti devastanti sulla reputazione degli sviluppatori.
- **Patch Manager (PM)**: questo ruolo è stato proposto per le organizzazioni che utilizzano del software (cioè tutte) e ha lo scopo di organizzare tutte le responsabilità relative alla gestione e all'applicazione delle patch nell'infrastruttura aziendale. Le sue attività principali comprendono:
  - Monitorare costantemente i bollettini di sicurezza dei fornitori e le fonti di intelligence sulle minacce per identificare tempestivamente le nuove patch disponibili.
  - Valutare quali patch siano applicabili all'ambiente aziendale, considerando i sistemi in uso e le loro configurazioni specifiche.
  - Stabilire le priorità di applicazione delle patch basandosi su criteri come CVSS, EPSS, criticità degli asset affetti e potenziale impatto sul business.
  - Coordinare il testing delle patch in ambienti di pre-produzione per verificarne la compatibilità e l'assenza di effetti collaterali indesiderati.
  - Pianificare e orchestrare il rollout delle patch nei vari ambienti produttivi, minimizzando le interruzioni di servizio.
  - Documentare tutte le attività di patching e mantenere un inventario aggiornato dello stato di patching di tutti i sistemi aziendali.
  - Gestire le eccezioni, ovvero i casi in cui una patch non può essere applicata immediatamente e devono essere implementate contromisure compensative.

  Questo ruolo è essenziale perché l'applicazione indiscriminata e non coordinata delle patch può causare interruzioni di servizio, incompatibilità e malfunzionamenti. Al contempo, ritardi nell'applicazione espongono l'organizzazione a rischi concreti di compromissione.

La separazione netta di questi ruoli riflette la diversità di prospettive tra sviluppatori e utenti. Il Software Flaw Manager si concentra sulla creazione di correzioni sicure ed efficaci, mentre il Patch Manager si focalizza sull'integrazione sicura di tali correzioni nell'ecosistema IT aziendale. Entrambi i ruoli richiedono competenze tecniche approfondite, ma anche capacità di coordinamento, comunicazione e gestione del rischio.

Le risorse limitate e la necessità di evitare interruzioni del servizio impongono la **prioritizzazione** e il **testing** delle patch. Intuitivamente, la priorità di correzione aumenta in funzione della gravità (CVSS/EPSS) e dell'importanza/valore dell'asset.

Il NIST ha proposto delle strategie di alto livello per razionalizzare il modo in cui il patch management viene affrontato:

- **Subsets**: consiglia di avere un elenco preciso delle categorie di macchine da "patchare". All'interno di ognuno di questi gruppi, conviene scegliere un piccolo sottoinsieme di macchine a cui iniziare ad applicare una patch, prima di procedere con l'intero insieme di macchine nella stessa categoria. L'obiettivo è testare la compatibilità e l'affidabilità della patch su pochi elementi ed evitare che eventuali problematiche causate dalla patch (malfunzionamenti, incompatibilità, regressioni) si propaghino sull'intera infrastruttura. Questo approccio incrementale permette di identificare tempestivamente eventuali effetti collaterali dannosi e di interrompere il rollout prima che l'intera categoria di sistemi venga compromessa, limitando il raggio d'azione di patch difettose.
- **Standard first**: questa strategia raccomanda di applicare le patch prioritariamente ai sistemi con configurazioni standard e ben documentate, che rappresentano tipicamente la maggior parte dell'infrastruttura aziendale. I vantaggi di questo approccio sono molteplici: i sistemi standard sono più facili da testare poiché le loro configurazioni sono note e riproducibili in ambienti di test; le patch per questi sistemi hanno maggiore probabilità di essere state testate dal fornitore in scenari tipici; eventuali problemi vengono rilevati più rapidamente grazie alla maggiore omogeneità dei sistemi; e soprattutto si riesce a coprire una larga porzione dell'infrastruttura in tempi relativamente brevi, riducendo significativamente l'esposizione complessiva al rischio. Questa strategia massimizza l'efficienza del processo, permettendo di proteggere rapidamente il maggior numero di asset con il minor sforzo.
- **Difficult ones later**: i sistemi complessi, *legacy*, o con configurazioni particolari (ad esempio sistemi SCADA, apparati medicali, dispositivi embedded, o sistemi con dipendenze software specifiche) vanno affrontati nelle fasi finali del processo di patching. Questi sistemi richiedono un'attenzione particolare perché potrebbero non supportare le patch più recenti senza modifiche sostanziali, potrebbero avere dipendenze critiche da versioni specifiche di librerie o componenti, i test di compatibilità richiedono ambienti specializzati e competenze tecniche approfondite, e il rischio di interruzione del servizio è spesso più elevato. Per questi sistemi è necessario pianificare collaudi manuali dettagliati, preparare piani di rollback specifici, talvolta aggirando i problemi o sviluppando contromisure compensative quando l'applicazione diretta della patch non è immediatamente possibile. Affrontando questi casi per ultimi, si garantisce che il personale tecnico possa dedicare il tempo e le risorse necessarie senza compromettere la protezione della maggioranza dei sistemi.
- **Cover all**: questa strategia finale sottolinea l'importanza di garantire una copertura completa ed esaustiva di tutti i sistemi aziendali, senza eccezioni. È fondamentale non trascurare categorie di dispositivi che potrebbero sembrare marginali ma che rappresentano potenziali vettori di attacco: macchine virtuali e virtualizzatori (che potrebbero essere dimenticate o considerate temporanee), dispositivi mobili (smartphone, tablet aziendali), firmware di dispositivi, in particolare di quelli di rete (router, switch, firewall), IoT e dispositivi embedded, sistemi di backup e ambienti di test o sviluppo. Ogni sistema non aggiornato rappresenta un potenziale punto di ingresso per gli attaccanti, che potrebbero sfruttarlo per penetrare nella rete aziendale e muoversi lateralmente verso asset più critici. È quindi necessario mantenere un inventario completo e aggiornato di tutti i dispositivi, implementare processi di discovery automatici per identificare nuovi asset e stabilire policy che obblighino l'applicazione delle patch anche ai sistemi considerati non critici.

1.8.2 Affidabilità delle Patch e Problematiche

Nonostante le patch siano fondamentali per la sicurezza, non sono esenti da rischi. Le patch possono introdurre nuove problematiche che impattano negativamente sui sistemi:

- **Incompatibilità software**: una patch può causare malfunzionamenti di applicazioni che dipendono da specifiche versioni di librerie o componenti modificati dalla correzione. Questo è particolarmente critico per software legacy o applicazioni personalizzate non più supportate.
- **Regressioni funzionali**: talvolta le patch introducono bug che compromettono funzionalità precedentemente operative, creando problemi operativi anche più gravi della vulnerabilità che dovevano correggere.
- **Degradazione delle prestazioni**: alcune patch, specialmente quelle che correggono vulnerabilità architetturali (come nel caso di Meltdown e Spectre), possono causare riduzioni significative delle performance del sistema.
- **Problemi di stabilità**: patch non sufficientemente testate possono causare crash di sistema, instabilità o comportamenti imprevedibili, richiedendo rollback urgenti.
- **Conflitti con altre patch**: in ambienti complessi, l'applicazione di multiple patch può generare conflitti o dipendenze circolari che compromettono l'intero sistema.

Come esempio più eclatante in tempi recenti di patch difettosa c'è sicuramente l'incidente CrowdStrike del luglio 2024, dove un aggiornamento difettoso del software di sicurezza CrowdStrike Falcon ha causato BSOD (Blue Screen of Death) massivi su milioni di sistemi Windows in tutto il mondo, bloccando aeroporti, ospedali e servizi critici[9]. Altri esempi possono includere patch di sicurezza per processori che hanno ridotto drasticamente le prestazioni (es. Meltdown), e correzioni che hanno reso inutilizzabili intere funzionalità, anche fondamentali, fino al rilascio di patch correttive delle patch stesse (es. Rete cellulare e TouchID in iOS 8.0.1[10]). Questi incidenti sottolineano l'importanza del testing accurato prima del deployment in produzione.

1.8.3 Evoluzione del Modello di Rilascio delle Patch

Storicamente, i fornitori di software rilasciavano patch **singolarmente**, non appena una vulnerabilità veniva scoperta e corretta. Questo approccio, apparentemente vantaggioso per minimizzare la *window of exposure*, creava numerosi problemi a livello aziendale:

- **Sovraccarico operativo**: gli amministratori di sistema dovevano costantemente monitorare, testare e applicare patch, dedicando la maggior parte del loro tempo al patch management invece che ad altre attività strategiche.
- **Interruzioni continue del servizio**: ogni patch richiedeva spesso un riavvio dei sistemi, causando interruzioni frequenti e impattando negativamente sulla disponibilità dei servizi.
- **Costi di testing elevati**: ogni singola patch doveva essere testata indipendentemente, moltiplicando i costi e i tempi necessari per la validazione.
- **Complessità di gestione**: tracciare quali patch erano state applicate a quali sistemi diventava un incubo amministrativo, con alto rischio di errori e dimenticanze.
- **Rischi di incompatibilità**: l'applicazione sequenziale di patch poteva creare situazioni in cui alcune patch dipendevano da altre non ancora installate o entravano in conflitto tra loro.

Per affrontare questi problemi, i fornitori hanno progressivamente adottato il modello delle **aggregated patch** (o *patch bundle*), rilasciando correzioni cumulative a cadenze regolari e prevedibili. Ad esempio, Microsoft ha introdotto il *Patch Tuesday*, rilasciando aggiornamenti di sicurezza il secondo martedì di ogni mese. Questo approccio presenta vantaggi significativi:

- **Prevedibilità**: le organizzazioni possono pianificare finestre di manutenzione programmate, riducendo l'impatto sul business.

[9] Wikipedia, 2025
[10] HWupgrade, 2014

- **Efficienza operativa**: un singolo ciclo di test e deployment sostituisce decine di interventi separati.
- **Testing integrato**: i fornitori possono testare l'interazione tra le varie correzioni prima del rilascio.
- **Riduzione dei riavvii**: invece di riavviare i sistemi più volte al mese, un singolo riavvio programmato può applicare tutte le patch.

Tuttavia, questo approccio presenta anche svantaggi: la *window of exposure* per le singole vulnerabilità può estendersi fino al prossimo rilascio programmato, e patch critiche potrebbero dover attendere settimane prima di essere distribuite (sebbene i fornitori mantengano la possibilità di rilasci immediati per vulnerabilità critiche attivamente sfruttate).

1.8.4 Enterprise Patch Management Tools

Gli **Enterprise Patch Management (EPM) Tools** sono soluzioni software centralizzate che automatizzano e orchestrano l'intero processo di patch management su larga scala. Strumenti come *Microsoft WSUS/SCCM*, *Red Hat Satellite*, *VMware vCenter Update Manager*, *ManageEngine Patch Manager*, e soluzioni cloud-based come *NinjaOne* o *Automox* offrono funzionalità avanzate:

- **Discovery automatica**: identificazione di tutti i sistemi nella rete e del loro stato di patching.
- **Distribuzione centralizzata**: deployment automatico delle patch su migliaia di endpoint da un'unica console.
- **Scheduling flessibile**: pianificazione di finestre di manutenzione diverse per gruppi di sistemi.
- **Reporting e compliance**: generazione di report dettagliati sullo stato del patching per audit e conformità normativa.
- **Rollback automatico**: capacità di annullare patch problematiche e ripristinare lo stato precedente.
- **Gestione multi-piattaforma**: supporto per Windows, Linux, macOS, e talvolta applicazioni di terze parti.

Nonostante i benefici evidenti, l'adozione di EPM introduce rischi significativi che devono essere attentamente gestiti:

- **Single point of failure**: se il server EPM viene compromesso, un attaccante ottiene un vettore perfetto per distribuire malware su tutta l'infrastruttura aziendale, trasformando uno strumento di sicurezza in un'arma devastante. Casi storici hanno dimostrato che attaccanti sofisticati mirano specificamente a questi sistemi.
- **Privileged access**: i tool EPM richiedono credenziali privilegiate per operare su tutti i sistemi gestiti. Se queste credenziali vengono rubate, l'attaccante ottiene accesso amministrativo all'intera rete.
- **Distribuzione di patch malevole**: un attaccante che compromette il repository delle patch o esegue attacchi man-in-the-middle può distribuire patch falsificate o backdoor mascherate da aggiornamenti legittimi.
- **Errori di configurazione**: configurazioni errate possono causare il deployment simultaneo di patch incompatibili su sistemi critici, provocando blocchi massicci.
- **Dipendenza dalla connettività**: sistemi remoti o isolati potrebbero non ricevere aggiornamenti se la connessione al server EPM è compromessa.
- **Complessità e manutenzione**: gli stessi EPM richiedono aggiornamenti, manutenzione e patching, creando un problema metaricorsivo del tipo "chi patcha il patcher?".

Per mitigare questi rischi, il NIST raccomanda diverse best practice, che non si discostano da quello che dovrebbe essere fatto con tutti gli altri sistemi critici o sensibili:

- **Hardening del server EPM**: isolamento in segmenti di rete protetti, applicazione di tutti i controlli di sicurezza, monitoraggio intensivo delle attività.

- **Autenticazione multi-fattore**: richiesta per tutti gli accessi amministrativi agli EPM.
- **Verifica dell'integrità**: uso di firme digitali e checksum per validare l'autenticità delle patch prima della distribuzione.
- **Segregazione dei privilegi**: limitazione dei permessi EPM al minimo indispensabile attraverso principi di least privilege.
- **Logging e auditing**: registrazione dettagliata di tutte le operazioni per investigazioni e forensics.
- **Deployment graduale**: uso di gruppi pilota e rollout progressivi anche con automazione, per identificare problemi prima che impattino l'intera infrastruttura.

Nonostante i rischi, l'analisi costi-benefici condotta dal NIST conclude che, se implementati correttamente con adeguate misure di sicurezza, gli EPM rappresentano una necessità per organizzazioni di medie e grandi dimensioni. I benefici in termini di riduzione della *window of exposure*, efficienza operativa, conformità normativa e riduzione dell'errore umano superano significativamente i rischi, purché questi ultimi siano attivamente mitigati attraverso le best practice di sicurezza.

Bibliografia

[1] David Lemire, Murugiah Souppaya e Kelley Dempsey. *Automation Support for Security Control Assessments: Volume 1 — Overview*. NIST Interagency or Internal Report (NISTIR) 8011, Volume 1. National Institute of Standards e Technology, apr. 2017. URL: https://nvlpubs.nist.gov/nistpubs/ir/2017/NIST.IR.8011-1.pdf.

Acronimi

ARP	Address Resolution Protocol	9
BSOD	Blue Screen of Death	16
CVE	Common Vulnerabilities and Exposures	5
CVSS	Common Vulnerability Scoring System	5
CWE	Common Weakness Enumeration	6
EPM	Enterprise Patch Management	17
EPSS	Exploit Prediction Scoring System	6
ICMP	Internet Control Message Protocol	9
IDS	Intrusion Detection System	7
ISCM	Information Security Continuous Monitoring	13
NVT	Network Vulnerability Test	10
OWASP	Open Web Application Security Project	13
PM	Patch Manager	14
RMF	Risk Management Framework	8
SFM	Software Flaw Manager	14
TCP	Transmission Control Protocol	9
UDP	User Datagram Protocol	9
ZDI	Zero Day Initiative	7